

Project 16: Chord + Replication

Scope and Objectives

The standard Chord protocol provides efficient, decentralized key lookups but lacks data persistence guarantees in the event of node failure. To address this, we will augment the core protocol with a successor-list replication strategy.

The primary objective is to build a peer-to-peer system where data remains available and durable despite node churn, specifically abrupt node failures. The implementation will be developed in Rust, utilizing the Tonic gRPC framework for inter-node communication and the Tokio runtime for asynchronous operations.

Deliverables

- Correct formation and maintenance of the Chord ring topology.
- Logarithmic time complexity for key lookups $O(\log N)$.
- Configurable r -way data replication across successor nodes.
- Data availability for reads and writes during single or multiple node failures, up to the replication factor limit.

System Architecture and Core Components

The system is composed of homogeneous peer nodes, each running the same binary. A node's architecture is defined by its data structures and the RPC services it exposes.

Node Data Structures

Each node in the Chord ring will maintain the following state:

- `id`: A k -bit identifier, derived from a SHA-1 hash of the node's IP address and port. This ID determines the node's position in the ring.
- `address`: The IP address and port for gRPC communication.
- `predecessor`: The address and ID of the node immediately preceding it on the ring. This is used for stabilization.
- `finger_table`: An array of size k . The i -th entry stores the address and ID of the first node that succeeds $(id + 2^{i-1}) \bmod 2^k$. This structure is essential for accelerating lookups.
- `successor_list`: An ordered list of the node's r immediate successors. This is critical for the replication strategy and for robustly maintaining the ring structure if the immediate successor fails.
- `key_value_store`: An in-memory hash map (`HashMap<Key, Value>`) that stores the key-value pairs for which this node is the primary coordinator or a replica.

gRPC Service Definition

Communication between nodes will be defined using a Protocol Buffers specification. The `ChordService` will expose RPCs to find successor, get predecessor, notify when a new predecessor has been found, and the usual put/get queries. It will also support a RPC to transfer keys to handle node joins.

Replication Strategy

To achieve fault tolerance, we will implement successor-list replication. For any given key k , the key-value pair will be stored on the coordinator node $s = \text{successor}(k)$ and on the $r - 1$ immediate successors of s . The integer r is the replication factor.

Write Path (Put operation)

1. A client sends a `Put(key, value)` request to an arbitrary entry-point node n .
2. Node n invokes `FindSuccessor(key)` to identify the coordinator node s .
3. Node n forwards the `Put` request to s .

Upon receiving the `Put` request, coordinator s :

1. Stores the `(key, value)` pair in its local `key_value_store`.
2. Iterates through its `successor_list` of size $r - 1$.
3. For each successor `succ_i` in the list, it sends a `PutReplica(key, value)` RPC (a variant of the `Put` RPC indicating it's a replicated write).

The operation is considered successful once the coordinator has stored the data. Replicas are updated asynchronously, favoring write availability over strict consistency.

Read Path (Get operation)

1. A client sends a `Get(key)` request to an entry-point node n .
2. Node n invokes `FindSuccessor(key)` to identify the coordinator node s .
3. Node n forwards the `Get` request to s .
4. If s responds successfully, the value is returned to the client.

If the RPC call to s fails, the requesting node n will re-attempt the `Get` request on the next node in the coordinator's successor list. This list can be obtained from the node that routed the request to the failed coordinator. This process repeats until a responsive replica is found or the list is exhausted.

Failure Handling and Data Reconciliation

Node failures are detected and handled by Chord's periodic stabilization tasks, which are extended to manage replica consistency.

- `stabilize()`: Each node n periodically checks if its successor s is alive. It also fetches s 's predecessor p . If p is a node between n and s , n updates its successor to p . This core logic is extended:
 - n also updates its `successor_list` by repeatedly asking its successor for its own successor list.
 - If n 's immediate successor fails, it connects to the next available node in its `successor_list` and promotes it to the primary successor.
 - n then initiates a re-replication process. It identifies the data it was replicating to the failed node and sends it to the new node that has entered its successor list.
- `join()`: When a new node n_{new} joins the ring between n and its successor s :
 - n_{new} identifies s as its successor.
 - n_{new} notifies s to set its predecessor to n_{new} .
 - n_{new} requests a transfer of all keys from s for which n_{new} is now the rightful coordinator.

Concurrently, n_{new} becomes part of the replication chain for data stored on its new predecessor, n . The stabilization process on n will detect n_{new} as its new successor and begin replicating the relevant key-value pairs to it.

This design ensures that data is dynamically re-replicated as the network topology changes, maintaining the desired level of redundancy and ensuring high data availability.